

## 03-源代码节选

### 1Claw 多智能体平台软件

#### 源代码节选

##### V1.0

---

**说明：** 以下为 1Claw 多智能体平台软件核心源代码节选，包含服务端（Go）和客户端（Dart/Flutter）的关键模块实现。

---

#### 前 30 页

##### 第 1-2 页：服务端入口 main.go

```
package main

import (
    "context"
    "flag"
    "fmt"
    "log"
    "net/http"
    "os"
    "os/exec"
```

```

    "os/signal"
    "path/filepath"
    "syscall"
    "time"

    "1claw-server/internal/agent"
    "1claw-server/internal/api"
    "1claw-server/internal/config"
    "1claw-server/internal/store"
    "1claw-server/internal/ws"
)

func main() {
    configPath := flag.String("config", "config.yaml", "Path to configura
    hermesHome := flag.String("hermes-home", "", "Path to Hermes home (~/)
    useMock := flag.Bool("mock", false, "Use MockBridge instead of real H
    flag.Parse()

    // Load configuration
    cfg, err := config.Load(*configPath)
    if err != nil {
        log.Fatalf("Failed to load config: %v", err)
    }

    // Resolve Hermes home — find via hermes binary symlink
    hh := *hermesHome
    if hh == "" {
        if hermesPath, err := exec.LookPath("hermes"); err == nil {
            if target, err := filepath.EvalSymlinks(hermesPath); err == n
                dir := filepath.Dir(target)
                for i := 0; i < 5; i++ {
                    if _, err := os.Stat(filepath.Join(dir, "profiles"));
                        hh = dir
                        break
                }
                parent := filepath.Dir(dir)
                if parent == dir {

```

```
                break
            }
            dir = parent
        }
    }
}
if hh == "" || hh == "." {
    hh = filepath.Join(os.Getenv("HOME"), ".hermes")
}
}
log.Printf("Hermes home: %s", hh)

// Discover user's Hermes profiles
userProfiles, err := config.DiscoverProfiles(hh)
if err != nil {
    log.Printf("Warning: could not discover profiles: %v", err)
}
cfg.Profiles = userProfiles

// Initialize WebSocket hub
hub := ws.NewHub()
go hub.Run()

// Initialize agent bridge
var bridge agent.Provider
if *useMock || !hermesAvailable() {
    if *useMock {
        log.Println("[bridge] Using MockBridge (--mock flag)")
    } else {
        log.Println("[bridge] Hermes not available, falling back to M
    }
    mb := agent.NewMockBridge()
    mb.LoadProfiles(cfg.Profiles)
    bridge = mb
} else {
    log.Println("[bridge] Starting per-profile Hermes agents ...")
    hb := agent.NewHermesBridge()
```

```
hb.HermesHome = hh
hb.Init()

// Spawn one process per profile
for _, p := range cfg.Profiles {
    if err := hb.SpawnProfile(p.ID); err != nil {
        log.Printf("Warning: failed to spawn agent %s: %v", p.ID,
        }
    }
}

// Wait for all to be ready (background)
go func() {
    for _, p := range cfg.Profiles {
        if err := hb.WaitReady(p.ID, 120*time.Second); err != nil {
            log.Printf("Warning: %s not ready: %v", p.ID, err)
        }
    }
}()
bridge = hb
}

// Start all agent profiles
ctx := context.Background()
if err := bridge.StartAll(ctx); err != nil {
    log.Fatalf("Failed to start agents: %v", err)
}
log.Printf("Loaded %d agent profiles", len(cfg.Profiles))

// Initialize SQLite chat store
chatDB := filepath.Join(hh, "1claw-chat.db")
chatStore, err := store.NewChatStore(chatDB)
if err != nil {
    log.Printf("Warning: could not open chat store: %v", err)
    chatStore = nil
} else {
    defer chatStore.Close()
}
```

```
// Create API server + WS handler
apiServer := api.NewServer(hub, bridge, cfg, hh)
wsHandler := api.NewWSHandler(hub, bridge, cfg, chatStore)
apiServer.Router.HandleFunc(cfg.Server.WSPath, wsHandler.ServeWS)

// Build the HTTP server
addr := fmt.Sprintf("%s:%d", cfg.Server.Host, cfg.Server.Port)
srv := &http.Server{
    Addr:         addr,
    Handler:      apiServer.Router,
    ReadTimeout:  15 * time.Second,
    WriteTimeout: 60 * time.Second,
    IdleTimeout:  60 * time.Second,
}

// Start server in a goroutine
go func() {
    log.Printf("1Claw Server starting on %s", addr)
    log.Printf("  WebSocket: ws://%s%s", addr, cfg.Server.WSPath)
    log.Printf("  API:      http://%s%s", addr, cfg.Server.APIPath)
    log.Printf("  Health:   http://%s/health", addr)
    if err := srv.ListenAndServe(); err != nil && err != http.ErrServ
        log.Fatalf("Server error: %v", err)
    }
}()

// Graceful shutdown
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("Shutting down server...")

for _, p := range bridge.GetProfiles() {
    if err := bridge.Stop(p.ID); err != nil {
        log.Printf("Error stopping profile %s: %v", p.ID, err)
    }
}
```

```

    }
    if hb, ok := bridge.(*agent.HermesBridge); ok {
        if err := hb.Close(); err != nil {
            log.Printf("Error closing bridge: %v", err)
        }
    }
    shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*t
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil {
        log.Fatalf("Server forced to shutdown: %v", err)
    }
    log.Println("Server stopped gracefully")
}

func hermesAvailable() bool {
    _, err := exec.LookPath("hermes")
    return err == nil
}

```

### 第 3 页: WebSocket Hub internal/ws/hub.go

```

package ws

import (
    "encoding/json"
    "log"
    "sync"

    "1claw-server/internal/model"
)

// Hub maintains the set of active WebSocket clients and broadcasts messa
type Hub struct {
    clients          map[*Client]bool
    clientByID       map[string]*Client
    register         chan *Client
    unregister       chan *Client
}

```

```
    broadcast      chan []byte
    profileUpdates chan []model.Profile
    mu              sync.RWMutex
}

func NewHub() *Hub {
    return &Hub{
        clients:      make(map[*Client]bool),
        clientByID:    make(map[string]*Client),
        register:      make(chan *Client),
        unregister:    make(chan *Client),
        broadcast:     make(chan []byte, 256),
        profileUpdates: make(chan []model.Profile, 64),
    }
}

func (h *Hub) Run() {
    for {
        select {
        case client := <-h.register:
            h.mu.Lock()
            h.clients[client] = true
            if client.ID != "" {
                h.clientByID[client.ID] = client
            }
            h.mu.Unlock()
            log.Printf("[ws] client connected: %s (total: %d)", client.ID)

        case client := <-h.unregister:
            h.mu.Lock()
            if _, ok := h.clients[client]; ok {
                delete(h.clients, client)
                if client.ID != "" {
                    delete(h.clientByID, client.ID)
                }
                close(client.Send)
            }
        }
    }
}
```

```
h.mu.Unlock()
log.Printf("[ws] client disconnected: %s (total: %d)", client

case message := <-h.broadcast:
h.mu.RLock()
for client := range h.clients {
select {
case client.Send <- message:
default:
h.mu.RUnlock()
h.mu.Lock()
delete(h.clients, client)
close(client.Send)
h.mu.Unlock()
h.mu.RLock()
}
}
h.mu.RUnlock()

case profiles := <-h.profileUpdates:
resp := model.WSResponse{
Type: "status",
Profiles: profiles,
}
data, err := json.Marshal(resp)
if err != nil {
log.Printf("[ws] marshal error: %v", err)
continue
}
h.mu.RLock()
for client := range h.clients {
select {
case client.Send <- data:
default:
}
}
h.mu.RUnlock()
```



```
    }  
  }  
}
```

#### 第 4-5 页: **WebSocket Client** internal/ws/client.go

```
package ws  
  
import (  
    "encoding/json"  
    "log"  
    "time"  
  
    "1claw-server/internal/model"  
    "github.com/gorilla/websocket"  
)  
  
const (  
    writeWait      = 10 * time.Second  
    pongWait       = 60 * time.Second  
    pingPeriod     = 30 * time.Second  
    maxMessageSize = 65536  
    sendBufSize    = 256  
)  
  
type Client struct {  
    ID          string  
    Hub         *Hub  
    Conn        *websocket.Conn  
    Send        chan []byte  
    ActiveProfile string  
    ConversationID string  
    OnChat      func(client *Client, msg model.WSMessage)  
    OnHistoryRequest func(client *Client)  
    OnMessage    func(client *Client, msg model.WSMessage)  
}
```

```
func NewClient(id string, hub *Hub, conn *websocket.Conn) *Client {
    return &Client{
        ID:      id,
        Hub:     hub,
        Conn:    conn,
        Send:    make(chan []byte, sendBufSize),
        OnChat:  defaultChatHandler,
    }
}

func (c *Client) ReadPump() {
    defer func() {
        c.Hub.unregister <- c
        c.Conn.Close()
    }()
    c.Conn.SetReadLimit(maxMessageSize)
    c.Conn.SetReadDeadline(time.Now().Add(pongWait))
    c.Conn.SetPongHandler(func(string) error {
        c.Conn.SetReadDeadline(time.Now().Add(pongWait))
        return nil
    })
    for {
        _, message, err := c.Conn.ReadMessage()
        if err != nil {
            if websocket.IsUnexpectedCloseError(err,
                websocket.CloseGoingAway, websocket.CloseNormalClosure) {
                log.Printf("[ws] read error: %v", err)
            }
            break
        }
        var msg model.WSMessage
        if err := json.Unmarshal(message, &msg); err != nil {
            log.Printf("[ws] invalid message: %v", err)
            c.sendError("invalid_message", "Invalid message format")
            continue
        }
        c.handleMessage(msg)
    }
}
```

```

    }
}

func (c *Client) WritePump() {
    ticker := time.NewTicker(pingPeriod)
    defer func() {
        ticker.Stop()
        c.Conn.Close()
    }()
    for {
        select {
        case message, ok := <-c.Send:
            c.Conn.SetWriteDeadline(time.Now().Add(writeWait))
            if !ok {
                c.Conn.WriteMessage(websocket.CloseMessage, []byte{})
                return
            }
            if err := c.Conn.WriteMessage(websocket.TextMessage, message); err != nil {
                log.Printf("[ws] write error: %v", err)
                return
            }
        case <-ticker.C:
            c.Conn.SetWriteDeadline(time.Now().Add(writeWait))
            if err := c.Conn.WriteMessage(websocket.PingMessage, nil); err != nil {
                return
            }
        }
    }
}

func (c *Client) handleMessage(msg model.WSMessage) {
    switch msg.Type {
    case "ping":
        c.sendPong()
    case "chat":
        if c.OnChat != nil {
            c.OnChat(c, msg)
        }
    }
}

```

```

    }
    case "switch_profile":
        c.ActiveProfile = msg.ProfileID
        log.Printf("[ws] client %s switched to profile %s", c.ID, msg.Pro
    case "get_status":
        if c.OnMessage != nil {
            c.OnMessage(c, msg)
        } else {
            c.sendPong()
        }
    case "get_history":
        if c.OnHistoryRequest != nil {
            c.OnHistoryRequest(c)
        } else {
            c.sendError("not_ready", "History not available")
        }
    default:
        if c.OnMessage != nil {
            c.OnMessage(c, msg)
        } else {
            c.sendError("unknown_type", "Unknown message type: "+msgType)
        }
    }
}
}

```

### 第 6-7 页: **Agent Bridge** 接口 internal/agent/bridge.go

```

package agent

import (
    "context"
    "fmt"
    "sync"
    "time"

    "1claw-server/internal/model"
)

```

```
type AgentBridge interface {
    Start(ctx context.Context, profile *model.Profile) error
    Stop(profileID string) error
    SendMessage(ctx context.Context, req model.ChatRequest) (string, error)
    CancelMessage(profileID, sessionID string) error
    GetStatus(profileID string) model.AgentStatus
}

type Provider interface {
    LoadProfiles(profiles []model.Profile)
    StartAll(ctx context.Context) error
    Start(ctx context.Context, profile *model.Profile) error
    Stop(profileID string) error
    SendMessage(ctx context.Context, req model.ChatRequest) (string, error)
    CancelMessage(profileID, sessionID string) error
    GetStatus(profileID string) model.AgentStatus
    GetAllStatus() []model.AgentStatus
    GetProfiles() []model.Profile
}

type AgentState struct {
    Profile *model.Profile
    Online  bool
    LastPing time.Time
    mu      sync.RWMutex
}

type MockBridge struct {
    agents map[string]*AgentState
    mu     sync.RWMutex
}

func NewMockBridge() *MockBridge {
    return &MockBridge{
        agents: make(map[string]*AgentState),
    }
}
```

```
}
```

```
func (b *MockBridge) LoadProfiles(profiles []model.Profile) {
    b.mu.Lock()
    defer b.mu.Unlock()
    for _, p := range profiles {
        prof := p
        b.agents[p.ID] = &AgentState{
            Profile: &prof,
            Online:   false,
            LastPing: time.Now(),
        }
    }
}
```

```
func (b *MockBridge) Start(ctx context.Context, profile *model.Profile) error {
    b.mu.Lock()
    defer b.mu.Unlock()
    state, exists := b.agents[profile.ID]
    if !exists {
        return fmt.Errorf("profile %s not found", profile.ID)
    }
    state.mu.Lock()
    state.Online = true
    state.LastPing = time.Now()
    state.Profile.Online = true
    state.mu.Unlock()
    return nil
}
```

```
func (b *MockBridge) StartAll(ctx context.Context) error {
    b.mu.RLock()
    profiles := make([]*model.Profile, 0, len(b.agents))
    for _, state := range b.agents {
        profiles = append(profiles, state.Profile)
    }
    b.mu.RUnlock()
}
```

```
    for _, p := range profiles {
        if err := b.Start(ctx, p); err != nil {
            return err
        }
    }
    return nil
}
```

第 8-9 页: **Hermes Bridge** internal/agent/hermes\_bridge.go (节选)

```
package agent

import (
    "bufio"
    "context"
    "encoding/json"
    "fmt"
    "log"
    "os"
    "os/exec"
    "path/filepath"
    "sync"
    "time"

    "1claw-server/internal/model"
)

type agentProcess struct {
    cmd      *exec.Cmd
    stdin    *json.Encoder
    stdout   *bufio.Scanner
    profileID string
    ready    bool
    mu       sync.Mutex
}

type HermesBridge struct {
```

```
mu            sync.RWMutex
agents        map[string]*agentProcess
spawnInfo     map[string]string
python        string
script        string
HermesHome    string
OnChatResponse func(profileID, content, msgID, sessionID string)
OnChatDispatched func(profileID, msgID, sessionID string)
}

func NewHermesBridge() *HermesBridge {
    return &HermesBridge{
        agents:    make(map[string]*agentProcess),
        spawnInfo: make(map[string]string),
    }
}

func (b *HermesBridge) Init() {
    b.python = filepath.Join(b.HermesHome, "hermes-agent", "venv", "bin",
    b.script = findScript("scripts/hermes_agent.py")
    if b.python == "" || !fileExists(b.python) {
        b.python = "python3"
    }
    log.Printf("[hermes] python=%s script=%s hermes-home=%s",
        b.python, b.script, b.HermesHome)
}

func (b *HermesBridge) SpawnProfile(pid string, hermesProfile ...string)
    profileName := pid
    if len(hermesProfile) > 0 && hermesProfile[0] != "" {
        profileName = hermesProfile[0]
    }
    b.mu.Lock()
    defer b.mu.Unlock()
    if _, exists := b.agents[pid]; exists {
        return nil
    }
}
```



```
isSpawn := pid != profileName
if isSpawn {
    b.spawnInfo[pid] = profileName
}
ap := &agentProcess{profileID: pid}
ctx := context.Background()
ap.cmd = exec.CommandContext(ctx, b.python, b.script)
ap.cmd.Env = os.Environ()
ap.cmd.Env = append(ap.cmd.Env, "HERMES_PROFILE="+profileName)
ap.cmd.Env = append(ap.cmd.Env, "HERMES_HOME="+b.HermesHome)

stdinPipe, err := ap.cmd.StdinPipe()
if err != nil {
    return fmt.Errorf("stdin pipe: %w", err)
}
ap.stdin = json.NewEncoder(stdinPipe)

stdoutPipe, err := ap.cmd.StdoutPipe()
if err != nil {
    return fmt.Errorf("stdout pipe: %w", err)
}
ap.stdout = bufio.NewScanner(stdoutPipe)
ap.stdout.Buffer(make([]byte, 0, 256*1024), 256*1024)
ap.cmd.Stderr = os.Stderr

if err := ap.cmd.Start(); err != nil {
    return fmt.Errorf("start agent %s: %w", pid, err)
}
b.agents[pid] = ap
log.Printf("[hermes] spawned agent %s (pid=%d)", pid, ap.cmd.Process.Pid)
go b.readAgentResponses(ap)
return nil
}
```

**第 10 页: WebSocket Handler** internal/api/ws\_handler.go (节选)

```
package api

import (
    "encoding/json"
    "log"
    "net/http"
    "sync/atomic"
    "time"

    "1claw-server/internal/agent"
    "1claw-server/internal/config"
    "1claw-server/internal/model"
    "1claw-server/internal/store"
    "1claw-server/internal/ws"
    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    ReadBufferSize: 1024,
    WriteBufferSize: 1024,
    CheckOrigin: func(r *http.Request) bool {
        return true
    },
}

type WSHandler struct {
    hub      *ws.Hub
    bridge    agent.Provider
    cfg       *config.Config
    chatStore *store.ChatStore
    clientCount atomic.Int32
}

func NewWSHandler(hub *ws.Hub, bridge agent.Provider,
    cfg *config.Config, chatStore *store.ChatStore) *WSHandler {
```

```
    return &WSHandler{
        hub:      hub,
        bridge:    bridge,
        cfg:       cfg,
        chatStore: chatStore,
    }
}
```

第 11 页: 数据模型 internal/model/types.go

```
package model
```

```
import "time"
```

```
type Profile struct {
    ID            string    `json:"id"`
    Name          string    `json:"name"`
    Emoji         string    `json:"emoji"`
    Description   string    `json:"description"`
    Color         string    `json:"color"`
    HermesProfile string    `json:"hermes_profile,omitempty"`
    Online        bool      `json:"online"`
    Status        string    `json:"status"`
    TasksQueue    int       `json:"tasks_queue"`
    IsSpawn       bool      `json:"is_spawn"`
    CreatedAt     time.Time `json:"created_at,omitempty"`
    UpdatedAt     time.Time `json:"updated_at,omitempty"`
}
```

```
type ChatMessage struct {
    ID            string    `json:"id"`
    ProfileID     string    `json:"profile_id"`
    Content       string    `json:"content"`
    Role          string    `json:"role"`
    Timestamp     time.Time `json:"timestamp"`
}
```

```
type WSMMessage struct {
    Type      string `json:"type"`
    ProfileID string `json:"profile_id,omitempty"`
    Content   string `json:"content,omitempty"`
    ID        string `json:"id,omitempty"`
    SessionID string `json:"session_id,omitempty"`
}

type WSResponse struct {
    Type      string `json:"type"`
    Profiles  []Profile `json:"profiles,omitempty"`
    ProfileID string `json:"profile_id,omitempty"`
    Content   string `json:"content,omitempty"`
    ID        string `json:"id,omitempty"`
    Code      string `json:"code,omitempty"`
    Message   string `json:"message,omitempty"`
    SessionID string `json:"session_id,omitempty"`
}
```

第 12 页: 配置加载 internal/config/config.go

```
package config

import (
    "fmt"
    "os"
    "path/filepath"
    "gopkg.in/yaml.v3"
)

type Config struct {
    Server  ServerConfig `yaml:"server"`
    Profiles []ProfileInfo `yaml:"profiles"`
}

type ServerConfig struct {
    Host      string `yaml:"host"`
}
```

```
    Port      int      `yaml:"port"`
    WSPath     string   `yaml:"ws_path"`
    APIPath    string   `yaml:"api_path"`
}

type ProfileInfo struct {
    ID          string `yaml:"id"`
    Name        string `yaml:"name"`
    Emoji       string `yaml:"emoji"`
    Description string `yaml:"description"`
    Color       string `yaml:"color"`
    HermesProfile string `yaml:"hermes_profile"`
}

func Load(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("read config: %w", err)
    }
    var cfg Config
    if err := yaml.Unmarshal(data, &cfg); err != nil {
        return nil, fmt.Errorf("parse config: %w", err)
    }
    return &cfg, nil
}
```

### 第 13-15 页: Flutter 应用入口 lib/main.dart

```
import 'dart:io';
import 'dart:async';
import 'dart:ui' as ui;

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:path_provider/path_provider.dart';
import 'app.dart';
import 'services/background_service.dart';
```



```
Future<void> clearCrashLog() async {
  try {
    final dir = await getApplicationDocumentsDirectory();
    final file = File('${dir.path}/crash_log.txt');
    if (await file.exists()) {
      await file.delete();
    }
  } catch (_) {}
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final appDir = await getApplicationDocumentsDirectory();
  _crashLogPath = '${appDir.path}/crash_log.txt';

  FlutterError.onError = (details) async {
    await _writeCrashLog('FlutterError', details.exception, details.stack);
    FlutterError.presentError(details);
  };

  PlatformDispatcher.instance.onError = (error, stack) {
    _writeCrashLog('PlatformDispatcher', error, stack);
    return true;
  };

  await NotificationService().initialize();
  BackgroundService().initialize();
  await BackgroundService().start();

  runApp(const ClawApp());
}

class MyHttpOverrides extends HttpOverrides {
  @override
  HttpClient createHttpClient(SecurityContext? context) {
    return super.createHttpClient(context)
      ..badCertificateCallback = (_, __, _) => true;
  }
}
```

```
}  
}
```

### 第 16-18 页：应用入口 lib/app.dart

```
import 'dart:async';  
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';  
import 'config/theme.dart';  
import 'providers/chat_provider.dart';  
import 'providers/font_settings_provider.dart';  
import 'providers/profiles_provider.dart';  
import 'providers/theme_provider.dart';  
import 'screens/home_screen.dart';  
import 'services/background_service.dart';  
import 'services/server_config_store.dart';  
import 'services/websocket_service.dart';  
  
class ClawApp extends StatefulWidget {  
  const ClawApp({super.key});  
  @override  
  State<ClawApp> createState() => _ClawAppState();  
}  
  
class _ClawAppState extends State<ClawApp> {  
  late final WebSocketService _wsService;  
  
  @override  
  void initState() {  
    super.initState();  
    _wsService = WebSocketService();  
  }  
  
  @override  
  void dispose() {  
    _wsService.dispose();  
    BackgroundService().stop();  
  }  
}
```



```

        super.dispose();
    }

    @override
    Widget build(BuildContext context) {
        return MultiProvider(
            providers: [
                ChangeNotifierProvider(create: (_) => ThemeProvider()),
                ChangeNotifierProvider(create: (_) => FontSettingsProvider()),
                ChangeNotifierProvider(
                    create: (_) => ProfilesProvider(_wsService),
                ),
                ChangeNotifierProvider(
                    create: (_) => ChatProvider(_wsService),
                ),
            ],
            child: _AppStartup(
                wsService: _wsService,
                child: Consumer2<ThemeProvider, FontSettingsProvider>(
                    builder: (context, themeProvider, fontProvider, _) => MaterialApp(
                        title: '1Claw',
                        debugShowCheckedModeBanner: false,
                        theme: AppTheme.light(
                            fontFamily: fontProvider.uiFont == 'System' ? '' : fontPr
                        darkTheme: AppTheme.dark(
                            fontFamily: fontProvider.uiFont == 'System' ? '' : fontPr
                        themeMode: themeProvider.themeMode,
                        home: const HomeScreen(),
                    ),
                ),
            ),
        );
    }
}

class _AppStartup extends StatefulWidget {
    final WebSocketService wsService;

```

```
final Widget child;
const _AppStartup({required this.wsService, required this.child});
@override
State<_AppStartup> createState() => _AppStartupState();
}
```

**第 19-21 页: WebSocket 服务 lib/services/websocket\_service.dart (节选)**

```
import 'dart:async';
import 'dart:convert';
import 'dart:math';
import 'package:flutter/foundation.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:web_socket_channel/web_socket_channel.dart';
import '../models/ws_message.dart';
```

```
class WebSocketService {
  WebSocketChannel? _channel;
  StreamSubscription? _channelSubscription;
  Timer? _heartbeatTimer;
  Timer? _reconnectTimer;
  Future<bool>? _connectFuture;
  int? _connectOperationId = 0;
  int? _connectFutureOperationId = 0;

  String _serverUrl = 'ws://localhost:8080/ws';
  bool _connected = false;
  bool _isConnecting = false;
  bool _disposed = false;
  int _reconnectAttempt = 0;
  bool _needsManualReconnect = false;

  void Function()? onNeedsManualReconnect;
  bool get needsManualReconnect => _needsManualReconnect;
  String? _clientId;
  String? conversationId;
  void Function(bool connected)? onConnectionChange;
```

```
final List<void Function(bool connected)> _connectionListeners = [];
final List<void Function(WsMessage)> _messageListeners = [];

void addConnectionListener(void Function(bool connected) listener) {
    _connectionListeners.add(listener);
}

void removeConnectionListener(void Function(bool connected) listener) {
    _connectionListeners.remove(listener);
}

void addMessageListener(void Function(WsMessage) listener) {
    _messageListeners.add(listener);
}

void removeMessageListener(void Function(WsMessage) listener) {
    _messageListeners.remove(listener);
}

WebSocketService();
String get serverUrl => _serverUrl;
bool get isConnected => _connected;
String? get clientId => _clientId;

void setServerUrl(String url) { _serverUrl = url; }

Future<String> _ensureClientId() async {
    if (_clientId != null) return _clientId!;
    final prefs = await SharedPreferences.getInstance();
    const key = '1claw_client_id';
    var id = prefs.getString(key);
    if (id == null || id.isEmpty) {
        id = 'client_${DateTime.now().millisecondsSinceEpoch}_${Random().nextDouble()};
        await prefs.setString(key, id);
    }
    _clientId = id;
}
```

```
    return id;
}

Future<bool> reconnect() async {
    if (_disposed) return false;
    _incrementConnectOperationId();
    _reconnectAttempt = 0;
    _needsManualReconnect = false;
    _reconnectTimer?.cancel();
    _reconnectTimer = null;
    await disconnect();
    return await connect();
}

Future<bool> connect() async {
    if (_disposed) return false;
    final opId = _incrementConnectOperationId();
    _isConnecting = true;
    _reconnectTimer?.cancel();

    try {
        final id = await _ensureClientId();
        final uri = Uri.parse('$_serverUrl?client_id=$id');
        final channel = WebSocketChannel.connect(uri);
        _channel = channel;
        _channelSubscription = channel.stream.listen(
            (data) => _onMessage(data),
            onError: (error) => _onError(error),
            onDone: () => _onDisconnected(),
            cancelOnError: false,
        );

        _connected = true;
        _isConnecting = false;
        _reconnectAttempt = 0;
        _startHeartbeat();
        _notifyConnectionChange(true);
    }
```

```
        return true;
    } catch (e) {
        _isConnecting = false;
        _scheduleReconnect();
        return false;
    }
}

void _onMessage(dynamic data) {
    try {
        final json = jsonDecode(data as String);
        final msg = WsMessage.fromJson(json);
        if (msg.type == 'pong') return;
        for (final listener in _messageListeners) {
            listener(msg);
        }
    } catch (e) {
        debugPrint('[ws] message parse error: $e');
    }
}

void _onError(dynamic error) {
    debugPrint('[ws] error: $error');
    _onDisconnected();
}

void _onDisconnected() {
    _connected = false;
    _stopHeartbeat();
    _notifyConnectionChange(false);
    if (!_disposed) _scheduleReconnect();
}
}
```

## 第 22-25 页：主界面 lib/screens/home\_screen.dart（节选）

```
import 'dart:async';
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:provider/provider.dart';
import '../config/constants.dart';
import '../models/agent_profile.dart';
import '../providers/chat_provider.dart';
import '../providers/profiles_provider.dart';
import '../services/api_service.dart';
import '../services/server_config_store.dart';
import '../widgets/agent_card.dart';
import '../widgets/connection_indicator.dart';
import '../widgets/chat_panel.dart';
import '../widgets/user_list_item.dart';
import '../widgets/toast.dart';
import '../widgets/profile_quick_panel.dart';
import 'chat_screen.dart';
import 'settings_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});
  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen>
  with WidgetsBindingObserver, TickerProviderStateMixin {
  bool _dialogShown = false;
  late bool _isWide;
  Timer? _resizeTimer;
  final TextEditingController _filterController = TextEditingController();
  String _filterQuery = '';
  String _sortBy = 'time';
  int _searchTriggerKey = 0;
```

```
late AnimationController _refreshSpinCtrl;
late AnimationController _gearSpinCtrl;
final Map<String, bool> _hoverStates = {};
final FocusNode _filterFocus = FocusNode();
bool _filterHasFocus = false;

@override
void initState() {
  super.initState();
  WidgetsBinding.instance.addObserver(this);
  _dialogShown = false;
  _refreshSpinCtrl = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 600),
  )..addListener(() => setState(() {}));
  _gearSpinCtrl = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 600),
  )..addListener(() => setState(() {}));
  _filterFocus.onKeyEvent = (node, event) {
    if (event is KeyDownEvent && event.logicalKey == LogicalKeyboardKey
        _filterController.clear();
        setState(() => _filterQuery = '');
        _filterFocus.unfocus();
        return KeyEventResult.handled;
    }
    return KeyEventResult.ignored;
  };
}
```

---

## 后 30 页

第 -29 ~ -28 页：主界面（后半部分）lib/screens/home\_screen.dart

主要包含筛选、布局、状态管理、设置对话框等功能的完整实现。

```
// Build method — renders the full home screen with profile grid
@override
Widget build(BuildContext context) {
  return Consumer2<ProfilesProvider, ChatProvider>(
    builder: (context, profilesProvider, chatProvider, child) {
      final profiles = profilesProvider.profiles;
      final statuses = profilesProvider.profileStatuses;
      final activeProfileId = chatProvider.activeProfileId;
      final filteredProfiles = _filterProfiles(profiles);
      return Scaffold(
        body: SafeArea(
          child: Column(
            children: [
              _buildTopBar(statuses),
              if (_isWide && activeProfileId != null) {
                Expanded(child: _buildWideLayout(
                  filteredProfiles, activeProfileId, chatProvider))
              } else {
                Expanded(child: _buildNarrowLayout(
                  filteredProfiles, chatProvider)),
              ],
            ),
          ),
        );
      },
    );
}

Widget _buildWideLayout(List<AgentProfile> profiles,
  String activeId, ChatProvider chatProvider) {
  final activeProfiles = profiles.where((p) => p.id == activeId).toList
  return Row(
```



```

        children: [
          SizedBox(
            width: 320,
            child: _buildProfileList(profiles),
          ),
          VerticalDivider(width: 1),
          Expanded(
            child: activeProfiles.isNotEmpty
              ? ChatPanel(profile: activeProfiles.first, showHeader: true
                : const Center(child: Text('Select a profile'))),
          ),
        ],
      );
    }

    Widget _buildNarrowLayout(
      List<AgentProfile> profiles, ChatProvider chatProvider) {
      return _buildProfileList(profiles);
    }

    Widget _buildProfileList(List<AgentProfile> profiles) {
      return ListView.builder(
        itemCount: profiles.length,
        itemBuilder: (context, index) {
          final profile = profiles[index];
          return AgentCard(
            profile: profile,
            onTap: () => _onProfileTap(profile),
          );
        },
      );
    }
  }

```

第 -27 ~ -26 页：后台保活服务 lib/services/background\_service.dart

```

import 'dart:async';
import 'dart:io';

```

```
import 'package:flutter/foundation.dart';
import 'package:flutter_background_service/flutter_background_service.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

class BackgroundService {
  static final BackgroundService _instance = BackgroundService._internal();
  factory BackgroundService() => _instance;
  BackgroundService._internal();

  FlutterBackgroundService? _service;
  bool _initialized = false;
  bool _running = false;

  void initialize() {
    if (_initialized) return;
    _service = FlutterBackgroundService();
    _initialized = true;
  }

  Future<void> start() async {
    if (_running) return;
    if (_service == null) return;

    await _service!.configure(
      iosConfiguration: IosConfiguration(
        autoStart: true,
        onForeground: () {},
        onBackground: () => true,
      ),
      androidConfiguration: AndroidConfiguration(
        onStart: _onStart,
        autoStart: true,
        isForegroundMode: true,
        notificationChannelId: '1claw_background',
        initialNotificationTitle: '1Claw',
        initialNotificationContent: 'Running in background',
        foregroundServiceNotificationId: 888,
```

```

    ),
  );
  _running = true;
}

@pragma('vm:entry-point')
static void _onStart(ServiceInstance service) {
  if (service is AndroidServiceInstance) {
    service.setForegroundNotificationInfo(
      title: '1Claw',
      content: 'Connected and ready',
    );
  }
  service.on('stopService').listen((event) {
    service.stopSelf();
  });
}

Future<void> stop() async {
  if (!_running) return;
  if (_service == null) return;
  _service!.invoke('stopService');
  _running = false;
}
}

```

第 -25 ~ -24 页: 通知服务 lib/services/notification\_service.dart

```

import 'package:flutter/foundation.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

class NotificationService {
  static final NotificationService _instance = NotificationService._internal();
  factory NotificationService() => _instance;
  NotificationService._internal();

  final FlutterLocalNotificationsPlugin _plugin =

```

```
FlutterLocalNotificationsPlugin();
bool _initialized = false;

Future<void> initialize() async {
  if (_initialized) return;
  const settings = InitializationSettings(
    android: AndroidInitializationSettings('@mipmap/ic_launcher'),
    iOS: DarwinInitializationSettings(),
    macOS: DarwinInitializationSettings(),
  );
  await _plugin.initialize(settings);
  _initialized = true;
}

Future<void> showNotification({
  required int id,
  required String title,
  required String body,
}) async {
  const androidDetails = AndroidNotificationDetails(
    '1claw_channel',
    '1Claw Messages',
    channelDescription: 'Notifications from 1Claw agents',
    importance: Importance.high,
    priority: Priority.high,
  );
  const details = NotificationDetails(
    android: androidDetails,
    iOS: DarwinNotificationDetails(),
  );
  await _plugin.show(id, title, body, details);
}
}
```

第 -23 ~ -22 页: 角色模型 lib/models/agent\_profile.dart

```
import 'package:flutter/painting.dart';

class AgentProfile {
  final String id;
  final String name;
  final String emoji;
  final String description;
  final String color;
  final String hermesProfile;
  bool online;
  String status;
  int tasksQueue;
  bool isPinned;
  bool isSpawn;

  AgentProfile({
    required this.id,
    required this.name,
    required this.emoji,
    required this.description,
    this.color = '#0078D7',
    this.hermesProfile = '',
    this.online = false,
    this.status = 'free',
    this.tasksQueue = 0,
    this.isPinned = false,
    this.isSpawn = false,
  });

  factory AgentProfile.fromJson(Map<String, dynamic> json) {
    return AgentProfile(
      id: json['id'] as String? ?? '',
      name: json['name'] as String? ?? '',
      emoji: json['emoji'] as String? ?? '☐',
      description: json['description'] as String? ?? '',
    );
  }
}
```

```
        color: json['color'] as String? ?? '#0078D7',
        hermesProfile: json['hermes_profile'] as String? ?? '',
        online: json['online'] as bool? ?? false,
        status: json['status'] as String? ?? 'free',
        tasksQueue: json['tasks_queue'] as int? ?? 0,
        isSpawn: json['is_spawn'] as bool? ?? false,
    );
}

Map<String, dynamic> toJson() => {
    'id': id,
    'name': name,
    'emoji': emoji,
    'description': description,
    'color': color,
    'hermes_profile': hermesProfile,
    'online': online,
    'status': status,
    'tasks_queue': tasksQueue,
    'is_spawn': isSpawn,
};

int get colorValue {
    final hex = color.replaceFirst('#', '');
    return int.parse('FF$hex', radix: 16);
}
}
```

第 -21 页：聊天消息模型 lib/models/chat\_message.dart

```
class ChatMessage {
    final String id;
    final String profileId;
    final String content;
    final String role;
    final DateTime timestamp;
```

```
ChatMessage({
  required this.id,
  required this.profileId,
  required this.content,
  required this.role,
  required this.timestamp,
});

factory ChatMessage.fromJson(Map<String, dynamic> json) {
  return ChatMessage(
    id: json['id'] as String? ?? '',
    profileId: json['profile_id'] as String? ?? '',
    content: json['content'] as String? ?? '',
    role: json['role'] as String? ?? 'user',
    timestamp: json['timestamp'] != null
      ? DateTime.parse(json['timestamp'] as String)
      : DateTime.now(),
  );
}
```

第 -20 ~ -18 页：角色卡片组件 lib/widgets/agent\_card.dart

```
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import '../config/constants.dart';
import '../models/agent_profile.dart';

class AgentCard extends StatefulWidget {
  final AgentProfile profile;
  final VoidCallback? onTap;
  const AgentCard({super.key, required this.profile, this.onTap});

  @override
  State<AgentCard> createState() => _AgentCardState();
}
```

```

class _AgentCardState extends State<AgentCard> {
  bool _hovered = false;

  Color get _profileColor => AppConstants.getProfileColor(widget.profile.

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return MouseRegion(
      onEnter: (_) => setState(() => _hovered = true),
      onExit: (_) => setState(() => _hovered = false),
      cursor: SystemMouseCursors.click,
      child: GestureDetector(
        onTap: widget.onTap,
        child: AnimatedContainer(
          duration: const Duration(milliseconds: 150),
          margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 4)
          decoration: BoxDecoration(
            color: _hovered
              ? _profileColor.withValues(alpha: 0.1)
              : Colors.transparent,
            borderRadius: BorderRadius.circular(8),
          ),
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 12, vertical:
          child: Row(
            children: [
              Text(widget.profile.emoji, style: const TextStyle(fontSiz
              const SizedBox(width: 12),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(widget.profile.name,
                      style: const TextStyle(
                        fontWeight: FontWeight.w600, fontSize: 15))
                    const SizedBox(height: 2),

```



```

        Text(widget.profile.description,
            style: TextStyle(
                fontSize: 12,
                color: theme.textTheme.bodySmall?.color),
            maxLines: 1, overflow: TextOverflow.ellipsis),
    ],
),
),
const SizedBox(width: 8),
Container(
    width: 10, height: 10,
    decoration: BoxDecoration(
        shape: BoxShape.circle,
        color: widget.profile.online
            ? AppConstants.onlineGreen
            : AppConstants.offlineGray,
    ),
),
],
),
),
),
),
);
}
}

```

第 -17 ~ -16 页：聊天面板 lib/widgets/chat\_panel.dart

```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/agent_profile.dart';
import '../providers/chat_provider.dart';
import 'chat_bubble.dart';
import 'thinking_indicator.dart';

class ChatPanel extends StatefulWidget {

```

```

    final AgentProfile profile;
    final bool showHeader;
    const ChatPanel({super.key, required this.profile, this.showHeader = true});

    @override
    State<ChatPanel> createState() => _ChatPanelState();
}

class _ChatPanelState extends State<ChatPanel> {
    final TextEditingController _inputController = TextEditingController();
    final ScrollController _scrollController = ScrollController();
    final FocusNode _inputFocus = FocusNode();

    @override
    void dispose() {
        _inputController.dispose();
        _scrollController.dispose();
        _inputFocus.dispose();
        super.dispose();
    }

    void _sendMessage(ChatProvider provider) {
        final text = _inputController.text.trim();
        if (text.isEmpty) return;
        provider.sendMessage(widget.profile.id, text);
        _inputController.clear();
        _scrollToBottom();
    }

    void _scrollToBottom() {
        WidgetsBinding.instance.addPostFrameCallback((_) {
            if (_scrollController.hasClients) {
                _scrollController.animateTo(
                    _scrollController.position.maxScrollExtent,
                    duration: const Duration(milliseconds: 200),
                    curve: Curves.easeOut,
                );
            }
        });
    }
}

```

```
    }  
  });  
}
```

---

第 -15 ~ -10 页：设置界面 lib/screens/settings\_screen.dart (节选)

```
import 'package:flutter/cupertino.dart';  
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';  
import 'package:file_picker/file_picker.dart';  
import 'dart:io';  
import '../config/constants.dart';  
import '../providers/font_settings_provider.dart';  
import '../providers/profiles_provider.dart';  
import '../providers/theme_provider.dart';  
import '../services/api_service.dart';  
import '../services/server_config_store.dart';  
import '../services/notification_service.dart';  
import '../widgets/font_picker_dialog.dart';  
import '../widgets/toast.dart';  
  
class SettingsScreen extends StatefulWidget {  
  const SettingsScreen({super.key});  
  @override  
  State<SettingsScreen> createState() => _SettingsScreenState();  
}  
  
class _SettingsScreenState extends State<SettingsScreen> {  
  late TextEditingController _wsUrlController;  
  late TextEditingController _apiUrlController;  
  bool _isSaving = false;  
  final TextEditingController _exportPasswordController =  
    TextEditingController();  
  final TextEditingController _exportConfirmPasswordController =  
    TextEditingController();
```

```

String? _exportFolderPath;
bool _isExporting = false;
String? _passwordError;
String? _confirmPasswordError;
String? _folderError;

bool get _exportValid =>
    _passwordError == null &&
    _confirmPasswordError == null &&
    _folderError == null &&
    _exportPasswordController.text.isNotEmpty &&
    _exportConfirmPasswordController.text.isNotEmpty &&
    _exportFolderPath != null;

@override
void initState() {
    super.initState();
    _wsUrlController =
        TextEditingController(text: AppConstants.defaultWsUrl);
    _apiUrlController =
        TextEditingController(text: AppConstants.defaultApiUrl);
    _loadServerConfig();
    _exportPasswordController.addListener(_validateExportForm);
    _exportConfirmPasswordController.addListener(_validateExportForm);
}
}

```

第 -9 ~ -5 页: 对话区组件 lib/widgets/chat\_bubble.dart

```

import 'package:flutter/material.dart';
import '../models/chat_message.dart';
import '../config/constants.dart';

class ChatBubble extends StatelessWidget {
    final ChatMessage message;
    final bool isUser;
    const ChatBubble({super.key, required this.message, this.isUser = false

```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Padding(
```

```
    padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
```

```
    child: Row(
```

```
      mainAxisAlignment:
```

```
        isUser ? MainAxisAlignment.end : MainAxisAlignment.start,
```

```
      crossAxisAlignment: CrossAxisAlignment.start,
```

```
      children: [
```

```
        if (!isUser) ...[
```

```
          const CircleAvatar(
```

```
            radius: 16,
```

```
            backgroundColor: Color(0xFF0078D7),
```

```
            child: Icon(Icons.smart_toy, size: 18, color: Colors.white)
```

```
          ),
```

```
          const SizedBox(width: 8),
```

```
        ],
```

```
      Flexible(
```

```
        child: Container(
```

```
          constraints: BoxConstraints(
```

```
            maxWidth: MediaQuery.of(context).size.width * 0.7,
```

```
          ),
```

```
          padding: const EdgeInsets.all(12),
```

```
          decoration: BoxDecoration(
```

```
            color: isUser
```

```
              ? const Color(0xFF0078D7)
```

```
              : Theme.of(context).cardColor,
```

```
            borderRadius: BorderRadius.only(
```

```
              topLeft: const Radius.circular(16),
```

```
              topRight: const Radius.circular(16),
```

```
              bottomLeft: isUser
```

```
                ? const Radius.circular(16)
```

```
                : const Radius.circular(4),
```

```
              bottomRight: isUser
```

```
                ? const Radius.circular(4)
```

```
                : const Radius.circular(16),
```

```

        ),
      ),
      child: Text(
        message.content,
        style: TextStyle(
          color: isUser ? Colors.white : null,
          fontSize: 15,
        ),
      ),
    ),
  ),
  if (isUser) ...[
    const SizedBox(width: 8),
    const CircleAvatar(
      radius: 16,
      backgroundColor: Color(0xFF4CAF50),
      child: Icon(Icons.person, size: 18, color: Colors.white),
    ),
  ],
],
),
);
}
}

```

第-4~-1页:配置与常量lib/config/constants.dart、lib/config/theme.dart

```

// constants.dart
import 'package:flutter/material.dart';

class AppConstants {
  static const String defaultWsUrl = 'ws://localhost:8080/ws';
  static const String defaultApiUrl = 'http://localhost:8080';
  static const Color primaryBlue = Color(0xFF0078D7);
  static const Color onlineGreen = Color(0xFF4CAF50);
  static const Color offlineGray = Color(0xFF9E9E9E);
  static const Color darkBg = Color(0xFF1E1E1E);
}

```

```
static const Color darkSurface = Color(0xFF2D2D2D);
static const Color darkCard = Color(0xFF383838);
static const Color lightBg = Color(0xFFF5F5F5);
static const Color lightSurface = Color(0xFFFFFFFF);
static const Color lightCard = Color(0xFFFFFFFF);
static const Map<String, Color> profileColors = {
  'assistant': Color(0xFF0078D7),
  'writer': Color(0xFF7B1FA2),
  'coder': Color(0xFF388E3C),
  'designer': Color(0xFFF57C00),
};
static Color getProfileColor(String id) {
  return profileColors[id] ?? primaryBlue;
}
}

// theme.dart
class AppTheme {
  static ThemeData light({String fontFamily = ''}) {
    return ThemeData(
      useMaterial3: false,
      fontFamily: fontFamily.isNotEmpty ? fontFamily : null,
      scaffoldBackgroundColor: AppConstants.lightBg,
      cardColor: AppConstants.lightCard,
      colorScheme: ColorScheme.fromSeed(seedColor: AppConstants.primaryBl
    );
  }

  static ThemeData dark({String fontFamily = ''}) {
    return ThemeData(
      useMaterial3: false,
      fontFamily: fontFamily.isNotEmpty ? fontFamily : null,
      scaffoldBackgroundColor: AppConstants.darkBg,
      cardColor: AppConstants.darkCard,
      colorScheme: ColorScheme.dark(
        primary: AppConstants.primaryBlue,
        surface: AppConstants.darkSurface,
```

```
        ),  
    );  
}  
}
```

---

文档版本: V1.0 更新日期: 2026 年 5 月